

TAD: Tipo De Dato Abstracto

surgue ya que los lenguajes tenían inconvenientes con la información que podían representar.

ABSTRACCION: representar algo a partir de sus características esenciales y suprimiendo la q no lo son. Principio de información oculta, no sabemos como esta representado internamente.

DOS TIPOS:

- 1) ABSTRACCION DE PROCESOS: posibilidad de hacer funciones o procedimientos (unidades) que nos permiten extender el lenguaje en terminos de que a partir de la definición de un método incorporo una nueva sentencia en mi código que implica un montón de instrucciones.
- 2) ABSTRACCION DE DATOS: permite incorporar y descubrir nuevos tipos en el lenguaje

Un TAD es una representación de datos y un conjunto de operaciones (funciones o procedimientos).

Un TAD es un tipo nuevo q le definimos operaciones q van a manipular esas instancias de tipo.

Los tipos de datos definidos por el usuario son TADS

SATISFACE:

- ENCAPSULAMIENTO: la representación del tipo y las operaciones permitidas se describen en una única unidad sintáctica. Refleja las abstracciones descubiertas en el diseño. Refleja los niveles de abstracción. Modificabilidad.
- OCULTAMIENTO: la representación y la implementación debe permanecer oculta.

La unidad de programación que puede implementar un TAD se llama CLASE.

ESPECIFICACION FORMAL:

Proporciona un conjunto de axiomas q describen el comportamiento de todas las operaciones. Tiene una parte sintáctica y una parte semantica.

TAD nombreDelTipo (valores que toma los datos del tipo)

Sintaxis //operaciones q se pueden realizar sobre el TAD

Operación(Tipo argumento,) -> Tipo resultado

...

Semantica //incorporo las definiciones pero con las instrucciones que hacen a la resolucio de las operaciones q le puedo aplicar al TAD.

Operación(valores particulares argumentos) => expresión resultado

Hay operaciones definidas por si mismas que se consideran constructores del TAD. Normalmente se elige como constructor la operación que inicializa.

EJEMPLO TAD: CONJUNTO

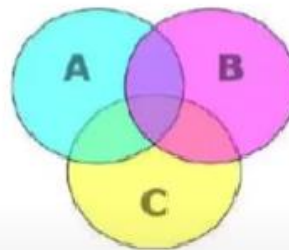
- *TAD Conjunto(colección de elementos sin duplicidades, pueden estar en cualquier orden, se usa para representar los conjuntos matemáticos con sus operaciones).*

Sintaxis

- *Conjuntovacio $\rightarrow$ Conjunto
- *Añadir(Conjunto, Elemento) $\rightarrow$ Conjunto
- Retirar(Conjunto, Elemento) $\rightarrow$ Conjunto
- Pertenece(Conjunto, Elemento) $\rightarrow$ boolean
- Esvacio(Conjunto) $\rightarrow$ boolean
- Cardinal(Conjunto) $\rightarrow$ entero
- Union(Conjunto, Conjunto) $\rightarrow$ Conjunto

Semántica $\forall e1, e2 \in \text{Elemento}$ y $\forall C, D \in \text{Conjunto}$

- Añadir(Añadir(C, e1), e1) $\Rightarrow$ Añadir(C, e1)
- Añadir(Añadir(C, e1), e2) $\Rightarrow$ Añadir(Añadir(C, e2), e1)
- Retirar(Conjuntovacio, e1) $\Rightarrow$ Conjuntovacio
- Retirar(Añadir(C, e1), e2) $\Rightarrow$ si $e1 = e2$ entonces Retirar(C, e2)
sino Añadir(Retirar(C, e2), e1)



EJEMPLO TAD: PILA EN ADA

- PILA de enteros (100)

ESPECIFICACION

```
package PILA IS
  type PILA limited private
  MAX: constant := 100
  function EMPTY (P:in PILA) return boolean
  procedure PUSH (P:inout PILA,ELE:in
    INTEGER)
  procedure POP (P: inout PILA)
  procedure TOP (P:inPILA) return INTEGER
private
  type PILA is
  vecpila : array (1..MAX) of INTEGER
  tope: INTEGER range 0..MAX:=0
end PILA
```

ENCAPSULA

OCULTA

IMPLEMENTACION

```
package body PILA is
  function EMPTY (P:in PILA) return boolean
  .....
end
  procedure PUSH (P:inout PILA,ELE:in INTEGER)
  .....
end
  procedure POP (P: inout PILA)
  .....
end
  procedure TOP (P:inPILA) return INTEGER
  .....
end
end PILA
```

OCULTA

Una CLASE es un tipo definido por el usuario que contiene la especificación de los datos que describen un objeto junto con la descripción de las acciones que un objeto conoce. ATRIBUTOS + METODOS

Agrega un SEGUNDO NIVEL DE ABSTRACCION que consiste en agrupar las clases en jerarquías. De forma que la clase hereda todas las propiedades de la superclase.

NO siempre se cumple con el ocultamiento de la información porq podemos tener atributos o métodos públicos.

SISTEMA DE TIPOS: evaluamos al lenguaje desde su sistema de tipos.

Es un conjunto de reglas usadas por un lenguaje para estructurar y organizar sus tipos.

El objetivo es escribir programas seguros. Por ende conocerlo nos permite conocer los aspectos semanticos del lenguaje de una mejor forma.

PROVEE MECANISMOS DE EXPRESION

- Expresar tipos intrínsecos o definir tipos nuevos.
- Asociar los tipos definidos con construcciones del lenguaje.

DEFINDE REGLAS DE RESOLUCION:

- Equivalencia de tipos ¿Dos valores distintos son del mismo tipo?
- Compatibilidad de tipos ¿Puedo usar el tipo en este contexto?
- Inferencia de tipos ¿Cuál tipo se deduce en el contexto?

Mientras mas flexible el lenguaje, mas complejo el sistema.

BUSCAMOS SEGURIDAD O BUSCAMOS FLEXIBILIDA? A mayor flexibilidad menor seguridad?

TIPADO FUERTE:

- Cuando especifica restricciones sobre como las operaciones que involucran valores de diferentes tipos pueden operarse. (definición muy generica).

TIPADO DEBIL:

- Cuando no especifica restricciones sobre como las operaciones que involucran valores de diferentes tipos pueden operarse.

COSAS A TENER EN CUENTA:

SISTEMA DE TIPOS – ESPECIFICACION:

- 1) [Tipo y tiempo de chequeo.](#)
- 2) [Reglas de equivalencia y conversión.](#)
- 3) [Reglas de inferencia de tipo.](#)
- 4) [Nivel de polimorfismo del lenguaje.](#)

1- Tipo y tiempo de chequeo:

Tipos de ligadura: ligadura de variable y tipo

- a- Tipado Estático: ligaduras en compilación. Para esto se puede exigir:
 - Se pueden utilizar tipos de datos predefinidos.
 - Todas las variables se declaran con un tipo asociado.
 - Todas las operaciones se especifican indicando los tipos de los operandos requeridos y el tipo del resultado.
- b- Tipado Dinámico: ligaduras en ejecución, provoca mas comprobaciones en tiempo de ejecución. Esto no implica que el lenguaje no es seguro.

Errores:



Lenguaje Fuertemente Tipado (type safety) respecto a su sistema de tipos: Definición

- Es fuertemente tipado si el compilador puede garantizar la ausencia de errores de tipo en los programas (Ghezzi)
- Es fuertemente tipado si el sistema de tipos impone restricciones que aseguran que no se producirán errores de tipo en ejecución. La intención es evitar los errores de aplicación y son tolerados los errores del lenguaje, detectados tan pronto como sea posible.

Python es Fuertemente Tipado y tiene tipado dinámico.
C es débilmente tipado y tiene tipado estático.
GOBSTONE Fuertemente Tipado y tipado estático.

2- Reglas de equivalencia y conversión:

- Tipo compatible: reglas semánticas que determinan si el tipo de un objeto es valido en un contexto particular. Ej los reales son compatibles con los enteros con respecto a la suma.
- Un lenguaje debe definir en que contexto el tipo Q es compatible con el tipo T. Si no lo hace dependerá del compilador
- El sistema de tipos define la compatibilidad

Reglas de Equivalencia y Conversión

```
type t = array [1..100] of integer
var x,y : array [1..100] of integer
z: array [1..100] of integer
w:t
v:t
```

w, x, y, z,v son del mismo tipo??

Compatibilidad < Nombre
Estructura

La compatibilidad/equivalencia se puede dar por nombre o por estructura:

- Equivalencia Por Nombre: dos variables son del mismo tipo si y solo si están declaradas juntas o si están declaradas con el mismo nombre de tipo.
- Equivalencia Por Estructura: dos variables son del mismo tipo si los componentes de su tipo son iguales.

En el ejemplo anterior T,X,Y,Z son del mismo tipo en cuanto a la estructura.

Un tipo es compatible con otro si:

- Es equivalente
- Se puede convertir

COERCION: significa convertir un valor de un tipo a otro. Para hacer esto, las reglas del lenguaje tiene que estar de acuerdo al tipo de los operandos y a la jerarquía q establece el mismo lenguaje.

ESTRATEGIAS PARA LA EQUIVALENCIA Y CONVERSION:

- 1) **WIDENING** (Ensachar): cada valor del dominio tiene su correspondiente valor en el rango. (Entero a Real). Pascal solo hace de entero a real.
- 2) **NARROWING** (Estrechar): cada valor del dominio puede no tener su correspondiente valor en el rango. En este caso algunos lenguajes producen un mensaje avisando la perdida de información (Real a Entero). En C depende del contexto y utiliza un sistema de coercion simple.
- 3) **CLAUSULA DE CASTING**: conversiones explicitas, se fuerza a que se convierta. Castear el valor.

3- Reglas de Inferencia y Nivel de Polimorfismo

La inferencia permite que el tipo de una entidad declarada se “infiera” en lugar de ser declarado. La inferencia puede realizarse de acuerdo al tipo de:

-Un Operando Predefinido (MOD solo para enteros):

Fun $f1(n,m)=(n \bmod m=0)$

-Un Operando (al ser 2 un numero, n va a tener q ser un numero):

Fun $f2(n)=(n*2)$

-Un Argumento (se infiere que el argumento es de tipo int por lo tanto si recibo un string lo proceso como un int):

Fun $f3(n) = n*n$

-El Tipo De resultado (si retorno un int entonces los parámetros también van a ser de tipo int):

Fun $f4(n)=(n*n)$

Nivel De Polimorfismo:

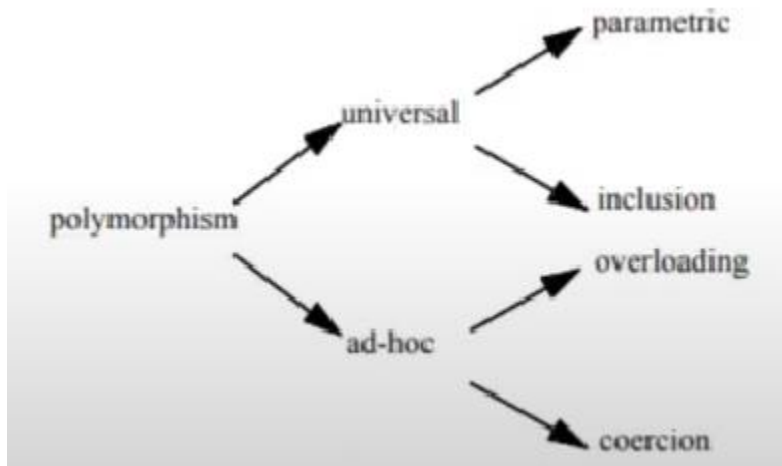
```
write(e,f(x)+1)
```

```
fun disjuntos(s1,s2:Conjunto):boolean;
```

- La función disjuntos deberá implementarse para cada tipo particular de conjunto?
- Las funciones read y write son “polimórficas”, pero no de forma pura (ya que el compilador infiere el tipo)

- Un lenguaje se dice mono-morfico si cada entidad se liga a un único tipo (estaticos). En un lenguaje de programación monomorfico la función disjuntos deberá implementarse para cada tipo particular de conjunto.

- Un lenguaje se dice polimórfico si las entidades pueden estar ligadas a mas de un tipo.
- Las variables polimórficas pueden tomar valores de diferentes tipos.
- Las operaciones polimórficas son funciones que aceptan operandos de varios tipos
- Los tipos polimórficos tienen operaciones polimórficas.



Diferentes tipos o grados de polimorfismo

Tipos De Polimorfismo:

-AD-HOC: Permite que una funcion se aplique a distintos tipos con un comportamiento sustancialmente diferente en cada caso.

-SOBRECARGA: se utiliza para referirse a conjuntos de abstracciones diferentes que están ligadas al mismo símbolo o identificador. Ejemplo el operado “+” para int los suma y para String los concatena. Operadores distintos cuyo simbolo es el mismo. Otro ej es la definición de un método, donde uno esta definido para recibir String y otro para recibir enteros.

-COERCIÓN: permite que un operador que espera un operando de un determinado tipo T puede aplicarse de manera segura sobre un operando de un tipo diferente al esperado. (castear). Puede ser implícita o explicita.

-UNIVERSAL: permite que una única operación se aplique uniformemente sobre un conjunto de tipos relacionados.

-PARAMETRICO: un tipo parametrizado es un tipo que tiene otros tipos como parámetros. Ej una lista de tipo String.

-POR_INCLUSION: es otra forma de polimorfismo universal que permite modelar subtipos y herencia. Si un tipo T define un conjunto de valores y operaciones, un subtipo T' de T

puede definirse como un subconjunto de los valores de T y el mismo conjunto de operaciones. El mecanismo de herencia permite definir una nueva clase derivada a partir de una clase base ya existente. Podría agregar atributos y comportamiento.